

数据结构

第三章 栈和队列

郭岗

计算机学院 网络技术中心

13370139202

guogang@bupt.edu.cn

教学内容

3.1 栈和队列的定义和特点

3.2 案例引入

3.3 栈的表示和操作的实现

3.4 栈与递归

3.5 队列的的表示和操作的实现

3.6 案例分析与实现

第3章 栈和队列



教学目标

1. 掌握栈和队列的**特点**，并能在相应的应用问题中正确选用
2. 熟练掌握栈的**两种存储结构**的基本操作实现算法，特别注意**栈满和栈空**的条件
3. 熟练掌握**循环队列和链队列**的基本操作实现算法，特别注意**队满和队空**的条件
4. 理解**递归算法**执行过程中栈的状态变化过程
5. 掌握**表达式求值方法**

1、概念

栈 (Stack)

1. 定义
2. 逻辑结构
3. 存储结构
4. 运算规则
5. 实现方式

队列 (Queue)

1. 定义
2. 逻辑结构
3. 存储结构
4. 运算规则
5. 实现方式



栈

1. 定义

只能在表的一端（栈顶）进行插入和删除运算的线性表

2. 逻辑结构

与线性表相同，仍为一对一关系

3. 存储结构

用顺序栈或链栈存储均可，但以顺序栈更常见

1、概念

4. 运算规则

只能在**栈顶**运算，且访问结点时依照**后进先出**（LIFO）或**先进后出**（FILO）的原则

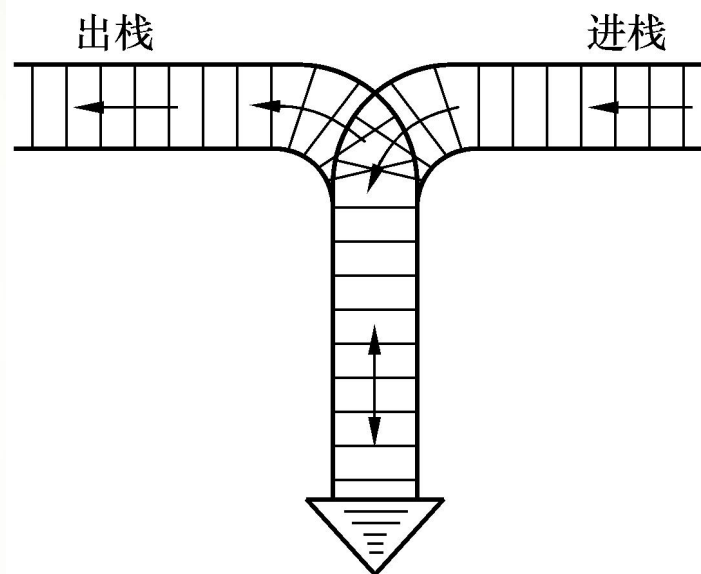
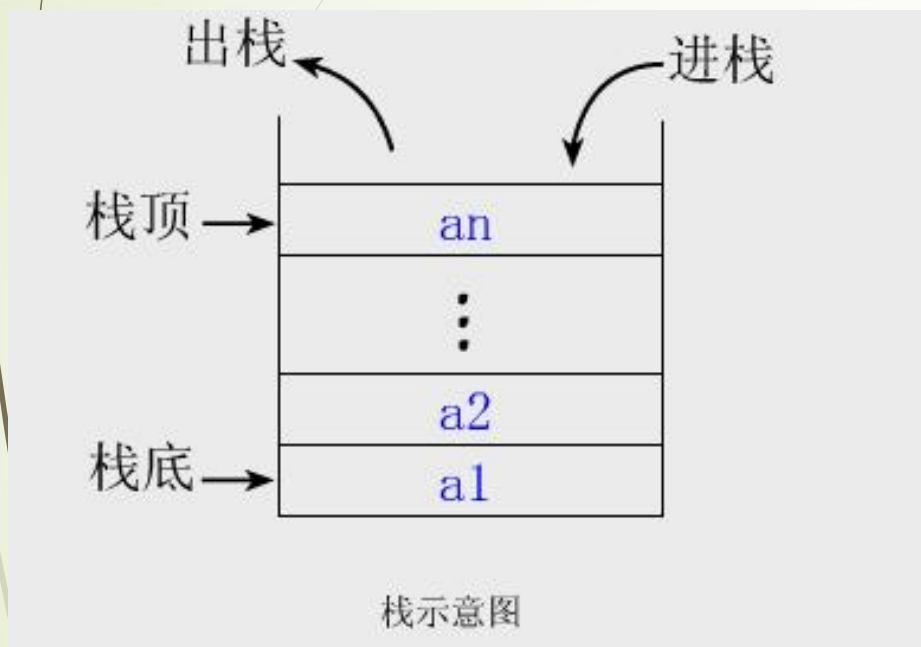
5. 实现方式

关键是编写**入栈**和**出栈**函数，具体实现依顺序栈或链栈的不同而不同

基本操作有**入栈**、**出栈**、**读栈顶元素值**、**建栈**、**判断栈满**、**栈空**等

1、概念

栈



用铁路调度站表示栈

五个子函数和main函数

```
int fun1(int n)
```

```
{
```

```
    if(n==0)
```

```
        return 1;
```

```
int fun5(int n)
```

```
{
```

```
    if (n==0)
```

```
        return 1;
```

```
    else
```

```
        return 1;
```

```
}
```

```
    return 1;
```

```
    else
```

```
        return n*fun5(n-1);
```

```
}
```

```
int fun2(int n)
```

```
{
```

```
    if(n==0)
```

```
int fun3(int n)
```

```
{
```

```
    if(n==0)
```

调用栈

```
#0  fun5 (n=0) at E:/temp/recursion/five.c:6
#1  0x00401332 in fun4 (n=1) at E:/temp/recursion/five.c:16
#2  0x0040135f in fun3 (n=2) at E:/temp/recursion/five.c:24
#3  0x0040138c in fun2 (n=3) at E:/temp/recursion/five.c:32
#4  0x004013b9 in fun1 (n=4) at E:/temp/recursion/five.c:40
#5  0x004013fb in main () at E:/temp/recursion/five.c:46
```

当被调用的fun5执行到n==0条件成立时，此时在调用栈中可以看到从函数fun1到函数fun5的依次调用次序

```
    return 1;
```

```
}
```

```
}
```



队列

1. 定义

只能在表的一端（**队尾**）进行插入，
在另一端（**队头**）进行删除运算的
线性表

2. 逻辑结构

与线性表相同，仍为**一对一**关系

3. 存储结构

用**顺序队列**或**链队**存储均可

1、概念

4. 运算规则

先进先出（FIFO）

5. 实现方式

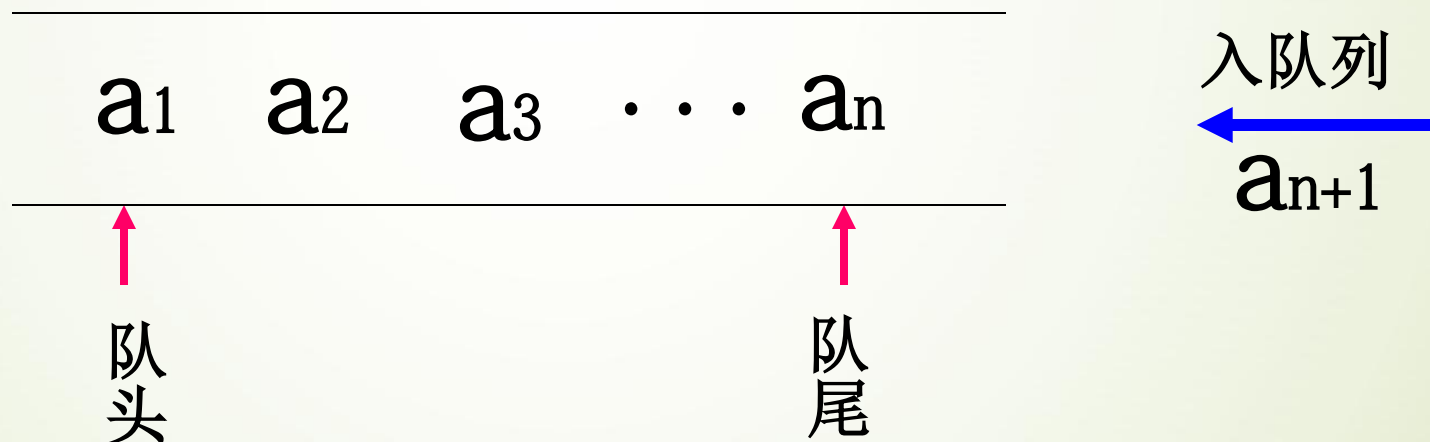
关键是编写入队和出队函数，具体实现依顺序队或链队的不同而不同

1、概念

队列是一种先进先出
(FIFO) 的线性表. 在
表一端插入, 在另一端
删除



$$q = (a_1, a_2, \dots, a_n)$$

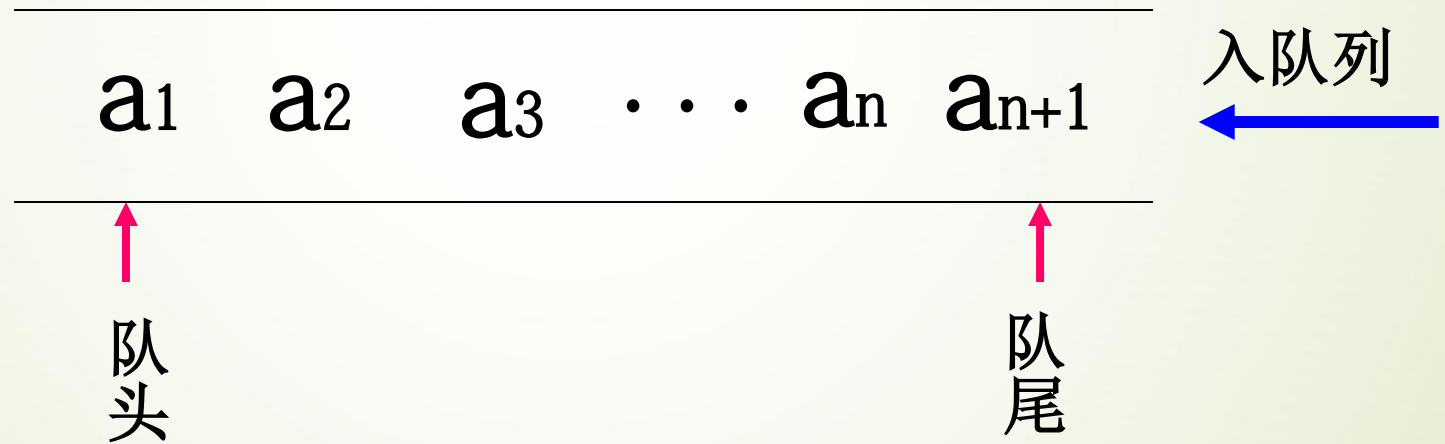


1、概念

队列是一种先进先出
(FIFO) 的线性表. 在
表一端插入, 在另一端
删除



$$q = (a_1, a_2, \dots, a_{n+1})$$

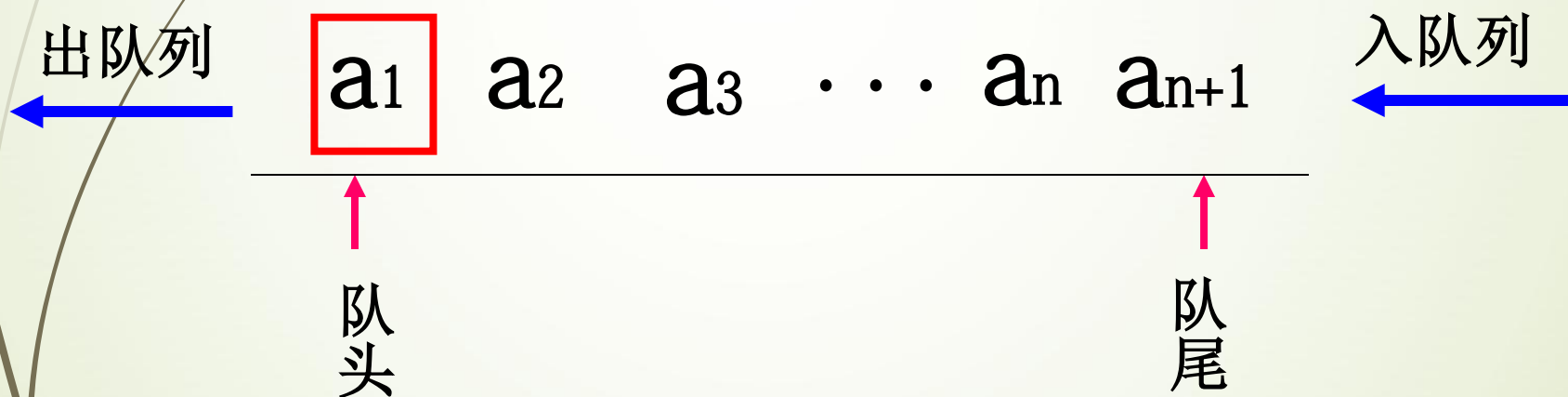


1、概念

队列是一种先进先出
(FIFO) 的线性表. 在
表一端插入, 在另一端
删除



$$q = (a_1, a_2, \dots, a_{n+1})$$



1、概念

队列是一种先进先出
(FIFO) 的线性表. 在
表一端插入, 在另一端
删除



$$q = (a_2, \cdots a_{n+1})$$

出队列
←
 a_1

a_2 a_3 $\cdots$ a_n a_{n+1}

入队列
←

↑
队头

↑
队尾

栈、队列与一般线性表的区别

1、概念

栈、队列是一种特殊（**操作受限**）的线性表
区别：仅在于**运算规则**不同

一般线性表

逻辑结构：一对一

存储结构：顺序表、链表

运算规则：**随机存取、
顺序存取即，任意位置存取**

栈

逻辑结构：一对一

存储结构：顺序栈、链栈

运算规则：**后进先出**

队列

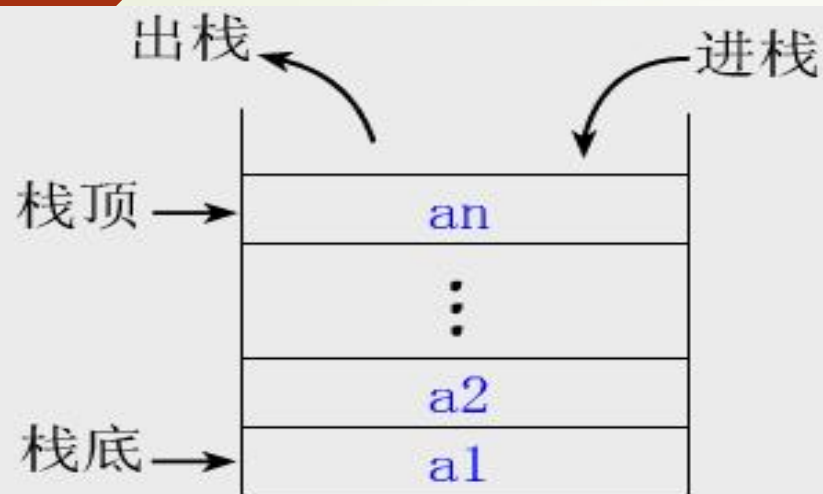
逻辑结构：一对一

存储结构：顺序队、链队

运算规则：**先进先出**

1、概念

栈的基本操作



栈示意图

“进” = 压入= PUSH()
“出” = 弹出= POP()

从一般线性表的角度来看，就是只能在表的尾部插入新元素，并且只能在表的尾部删除老元素；
或只能在表的头部插入新元素，并且只能在表的头部删除老元素；

练习

1、概念

若已知一个栈的入栈序列是 $1, 2, 3, \dots, n$, 其输出序列为 $p_1, p_2, p_3, \dots, p_n$, 若 $p_1=n$, 则 p_i 为

C

- A. i
- B. $n-i$
- C. $n-i+1$
- D. 不确定

练习

1、概念

如果一个栈的输入序列为123456，能否得到435612和135426的出栈序列？

输出：

4 3 5 6 2 1

PUSH (1)

PUSH (2)

PUSH (3)

PUSH (4)

POP ()

POP ()

PUSH (5)

POP ()

PUSH (6)

POP ()

POP ()

POP ()

练习

1、概念

如果一个栈的输入序列为123456，能否得到435612和135426的出栈序列？

435612中到了12顺序不能实现；

135426可以实现。

1、概念

队列的基本操作

“进” = 入队列 = EnQueue ()

“出” = 出队列 = DeQueue ()



从一般线性表的角度来看，就是只能在表的尾部插入新元素，并且只能在表的头部删除老元素；
或只能在表的头部插入新元素，并且只能在表的尾部删除老元素；

练习

1、概念

若已知一个队列的入队列序列是1, 2, 3, $\dots$, n, 其输出序列为 p_1 , p_2 , p_3 , $\dots$, p_n , 则 p_i 为

A

A. i

B. $n-i$

C. $n-i+1$

D. 不确定

练习

1、概念

如果一个队列的输入序列为123456，
其输出顺序是什么？

永远是123456

在线性表的基础上实现栈和队列的功能

1、概念

一般线性表的两个重要操作（函数）

```
Status ListTailInsert(List &L, ElemType e); //尾插入  
Status ListDelete(SqList &L, int i, ElemType &e );
```

栈

运算规则：后进先出，即尾入尾出

```
Status Push( Stack &S, ElemType e)  
{  
    return ListTailInsert(S, e);  
}  
Status Pop( SqStack &S, ElemType &e)  
{  
    return ListDelte_Sq(S, S.Length, e);  
}
```

队列

运算规则：先进先出，即尾入头出

```
Status EnQueue( Queue &Q, ElemType e)  
{  
    return ListTailInsert(Q, e);  
}  
Status DeQueue( SqStack &S, ElemType &e)  
{  
    return ListDelte_Sq(S, 1, e);  
}
```

栈和队列，选择线性表的那种实现方式更优？

利用线性表的基本方法（函数）实现后的讨论

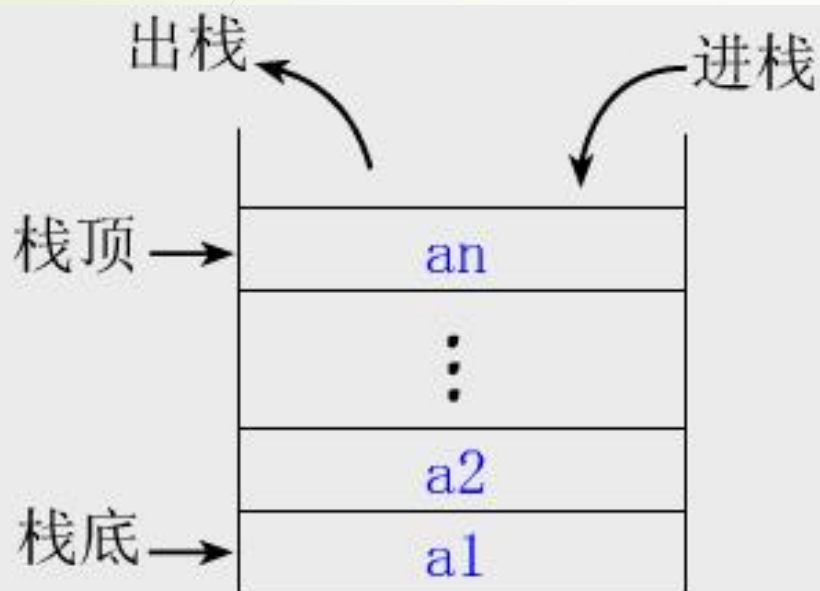
	栈		队列	
	栈顶为表头	栈顶为表尾	队头为表头	队头为表尾
顺序存储	PUSH, POP,	PUSH, POP,	EnQueue, DeQueue 用下标直接访问访问	EnQueue, DeQueue
链式存储	PUSH, POP,	PUSH, POP,	EnQueue, DeQueue	EnQueue, DeQueue

利用线性表的基本方法（函数）实现后的讨论

	栈		队列	
	栈顶为表头	栈顶为表尾	队头为表尾	队头为表头
顺序存储	PUSH, 在下标为1的位置插入, 顺序后移其他元素 时间复杂度$O(n)$ POP, 在下标为1的位置删除, 顺序前移其他元素 时间复杂度$O(n)$	PUSH, 用下标直接访问线性表的表尾 时间复杂度$O(1)$ POP, 用下标直接访问线性表的表尾 时间复杂度$O(1)$	EnQueue, 在下标为1的位置插入, 顺序后移其他元素 时间复杂度$O(n)$ DeQueue 用下标直接访问访问表尾 时间复杂度$O(1)$	EnQueue, 用下标直接访问访问表尾, 时间复杂度$O(1)$ DeQueue 在下标为1的位置删除, 顺序前移其他元素 时间复杂度$O(n)$
链式存储	PUSH, 头插法 时间复杂度$O(1)$ POP, 头指针后面删除 时间复杂度$O(1)$	PUSH, 尾插法, 先移动到表尾 时间复杂度$O(n)$ POP, 尾删除, 先移动到表尾前, 时间复杂度$O(n)$	EnQueue, 头插法 时间复杂度$O(1)$ DeQueue 尾删除, 先移动到表尾前, 时间复杂度$O(n)$	EnQueue, 尾插法, 先移动到表尾, 时间复杂度$O(n)$ DeQueue 头指针后面删除 时间复杂度$O(1)$

1、概念

3.3 栈实现方式的优化



栈示意图

“进” = 压入 = PUSH ()

“出” = 弹出 = POP ()

定义在栈结构上的基本运算

1、概念

- | | |
|----------------|-----------------------------------|
| (1) 生成空栈操作 | InitStack(&S) |
| (2) 销毁栈 | DestroyStack(&S) |
| (2) 判栈空函数 | StackEmpty (S) |
| (3) 数据元素入栈操作 | Push (&S,e) |
| (4) 数据元素出栈函数 | Pop (&S,&e) |
| (5) 取栈顶元素函数 | GetTop(S) //读栈顶元素，栈不变化 |
| (6) 置栈空操作 | ClearStack(&S) //清空栈元素 |
| (7) 求当前栈元素个数函数 | StackLength(S) |

用采用数组定义方式的顺序表，实现栈

顺序栈 实现方式1

由于栈是运算受限的线性表，因此线性表的存储结构对栈也适用。

和顺序表相似，**固定大小的顺序栈**的类型描述如下：

```
#define MAXSIZE 1024
typedef struct
{
    datatype data[MAXSIZE];
    int top;
}SeqStack;
SeqStack *s;    //定义一个指向顺序栈的指针
```

- 通常顺序表的0下标端设为栈底，**入栈时top向上增长；出栈时top向下减少**
- 初始化栈顶指针s->top值为-1，表示空栈

顺序栈 实现方式1

(1) 置空栈（初始化栈）
首先建立栈空间，然后
初始化栈顶指针。

重要基本操作的算法实现

1.3 初始化线性表L（返回一个线性表指针）

```
SqList * InitList_Sq(unsigned uListSize )  
//构造一个内容为空，长度为uListSize的顺序表L
```

```
SeqStack *Init_SeqStack()  
{  
    SeqStack *s;  
    s=(SeqStack *) malloc(sizeof(SeqStack)); /*申请栈空间*/  
  
    if (!s)  
    {  
        printf("空间不足"\n);  
        return NULL; /*未申请到足够大的存储空间，返回空指针*/  
    }  
    else  
    {  
        s->top=-1; /*初始化栈顶指针*/  
        return s; /*申请到栈空间，返回栈空间地址*/  
    }  
}
```

用采用数组定义方式的顺序表，实现栈

顺序栈 实现方式1

参考顺序存储的非固定大小的线性表定义，**初始化时分配内存的**类型描述如下：

```
typedef struct
{
    datatype *data; //data在初始化时指向分配的内存
    int top;
    int nMaxStackSize
}SeqStack;

SeqStack *s; //定义一个指向顺序栈的指针
```

- 通常顺序表的0下标端设为栈底，**入栈时top向上增长；出栈时top向下减少**
- 初始化栈顶指针s->top值为-1，表示空栈

(1) 置空栈（初始化栈）
首先建立栈空间，然后
初始化栈顶指针。

顺序栈 实现方式1

```
SeqStack *Init_SeqStack(int nMaxStacksize)
{
    SeqStack *s;
    s=(SeqStack *) malloc(sizeof(SeqStack)); /*申请栈空间*/

    if (!s)
    {
        return NULL;        /*未申请到足够大的存储空间，返回空指针*/
    }
    else
    {
        s->data = (datatype*)malloc(sizeof(datatype)*nMaxStackSize);
        if(!s->data)
        {
            free(s); return NULL;
        }
        s->top=-1;           /*初始化栈顶指针*/
        s->nMaxStackSize = nMaxStacksize;
        return s;           /*申请到栈空间，返回栈空间地址*/
    }
}
```

重要基本操作的算法实现

1.3 初始化线性表L（返回一个线性表指针）

```
SqList * InitList_Sq(unsigned uListSize )
//构造一个内容为空，长度为uListSize的顺序表L
```

//有else的代码结构

(1) 置空栈（初始化栈）
首先建立栈空间，然后
初始化栈顶指针。

顺序栈 实现方式1

```
SeqStack *Init_SeqStack(int nMaxStacksize)
{
    SeqStack *s;
    s=(SeqStack *) malloc(sizeof(SeqStack)); /*申请栈空间*/

    if (!s)
    {
        return NULL;        /*未申请到足够大的存储空间，返回空指针*/
    }

    s->data = (datatype*)malloc(sizeof(datatype)*nMaxStackSize);
    if(!s->data)
    {
        free(s); return NULL;
    }
    s->top=-1;                /*初始化栈顶指针*/
    s->nMaxStackSize = nMaxStacksize;
    return s;                /*申请到栈空间，返回栈空间地址*/
}
```

重要基本操作的算法实现

1.3 初始化线性表L（返回一个线性表指针）

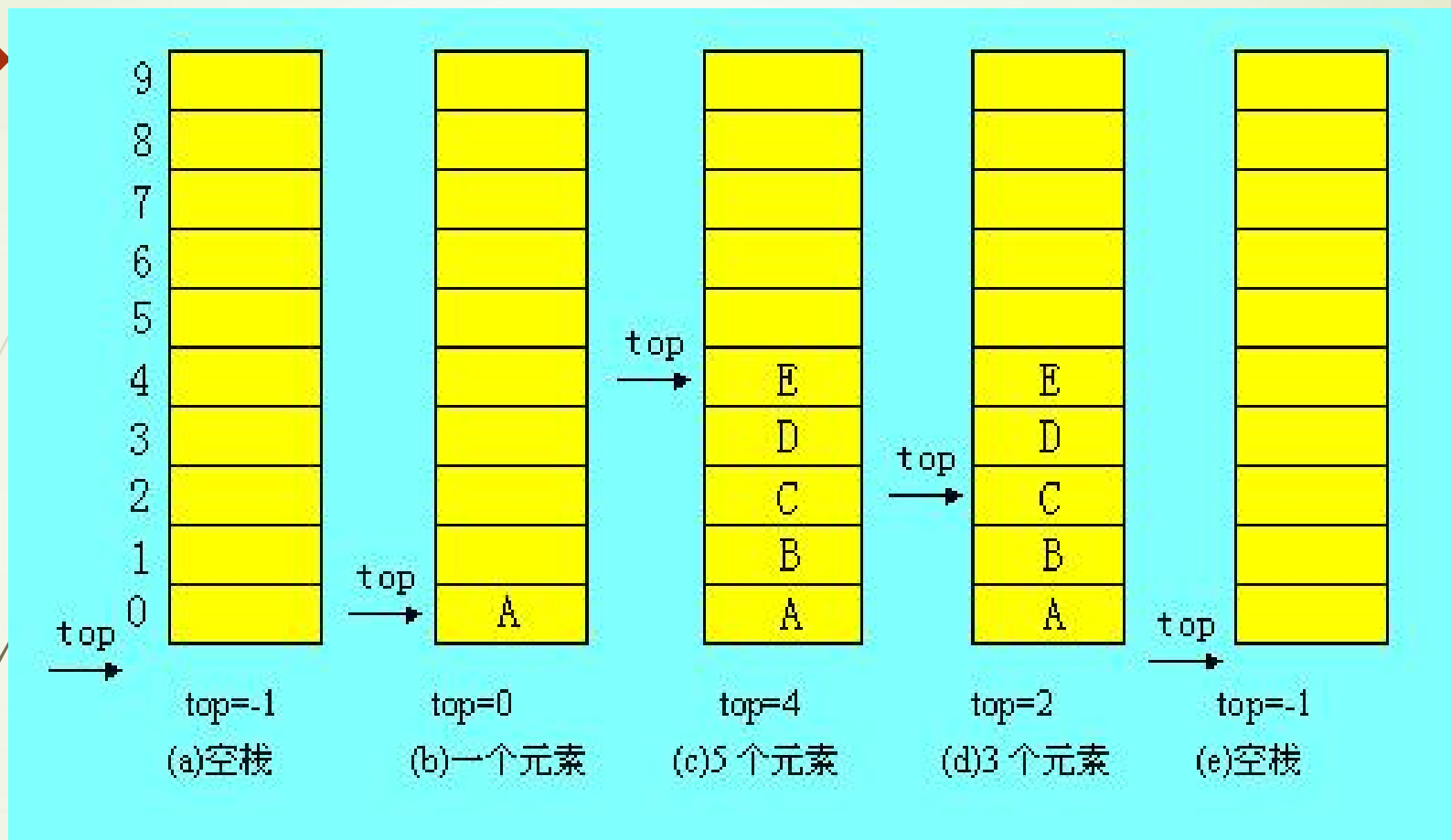
```
SqList * InitList_Sq(unsigned uListSize)
//构造一个内容为空，长度为uListSize的顺序表L
```

//无else的代码结构

Q：什么时候可以在if语句使用时采用这种无else的结构？

此数据结构下，栈操作的示意图：

顺序栈
实现方式1



(a): 空栈;

(c): A、B、C、D、E 5个元素依次入栈之后;

(d): 在图(c)之后E、D相继出栈, top 指针已经指向了新的栈顶, 则元素D、E已不属于栈中元素。

(2) 判空栈

顺序栈 实现方式1

检查栈顶指针的值。

【算法】判栈空算法（采用固定长度栈的数据存储结构）

```
int Empty_SeqStack(SeqStack *s)
{
    if (s->top == -1)
        return 1;           /*空栈返回1*/
    else
        return 0;
}
```

(3) 入栈

顺序栈 实现方式1

尾插入。

【算法】入栈算法（采用固定长度栈的数据存储结构）

```
int Push_SeqStack(SeqStack *s,datatype x)
{
    if ( s->top == MAXSIZE-1 )
        return 0;    /*栈满不能入栈，返回错误代码0*/
    else
    {
        s->top++;      /*栈顶指针向上移动*/
        s->data[s->top]=x; /*将x至入新的栈顶*/
        return 1;      /*入栈成功，返回成功代码1 */
    }
}
```

(4) 出栈

顺序栈 实现方式1

尾弹出。

【算法】出栈算法（采用固定长度栈的数据存储结构）

```
int Pop_SeqStack(SeqStack *s, datatype *x)
{
    /*从栈中弹出栈顶元素*/
    /*栈顶元素可以通过对指针类型参数x指向的内存赋值而返回*/
    if (Empty_SeqStack(s))
    {
        return 0;                /*栈空不能出栈，返回错误代码0*/
    }
    else
    {
        *x=s->data[s->top];      /*保存栈顶元素值*/
        s->top--;                /*栈顶指针向下移动*/
        return 1;                /*返回成功代码1 */
    }
}
```

(5) 取栈顶元素

顺序栈 实现方式1

线性表按位置存取。

【算法】取栈顶元素算法（采用固定长度栈的数据存储结构）

```
int Top_SeqStack(SeqStack *s, datatype *x)
{
    /*仅是读取栈顶元素，栈顶指针不发生变化*/
    /*栈顶元素可以通过对指针类型参数x指向的内存赋值而返回*/
    if (Empty_SeqStack(s))
    {
        return 0; /*栈空无元素*/
    }
    else
    {
        *x=s->data[s->top];

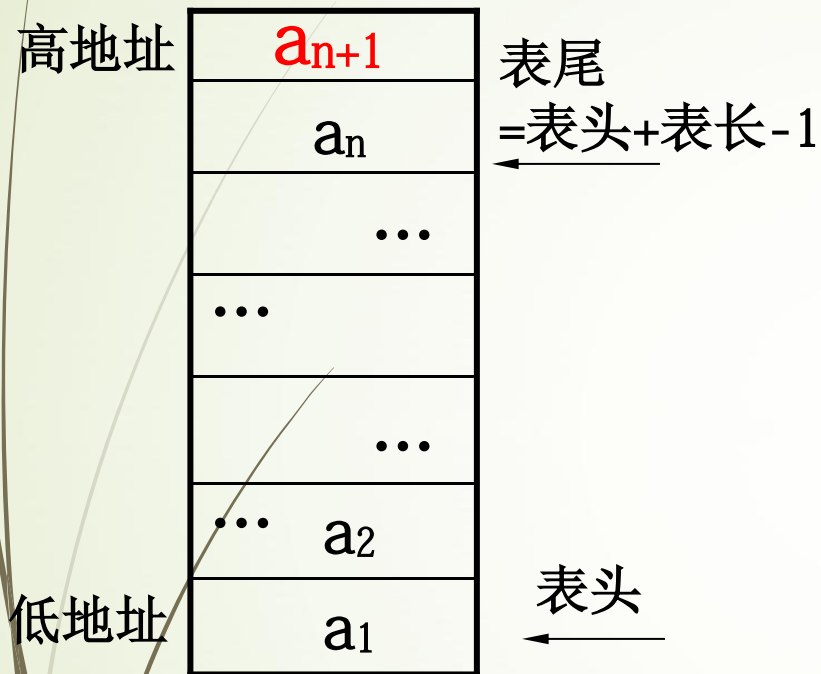
        return 1;
    }
}
```

用采用指针方式表示的顺序表，实现栈

顺序栈

实现方式2

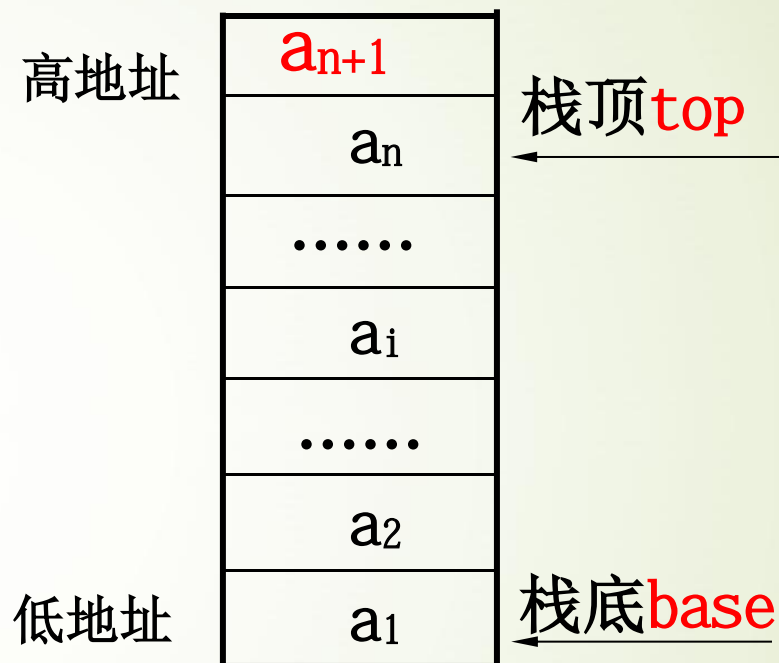
顺序表V[n]



写入表尾: $v[\text{表头} + \text{表长} - 1 + 1] = a_{n+1}$

读出表尾: $x = v[\text{表头} + \text{表长} - 1]$

顺序栈S



压入: $\text{PUSH}(a_{n+1})$

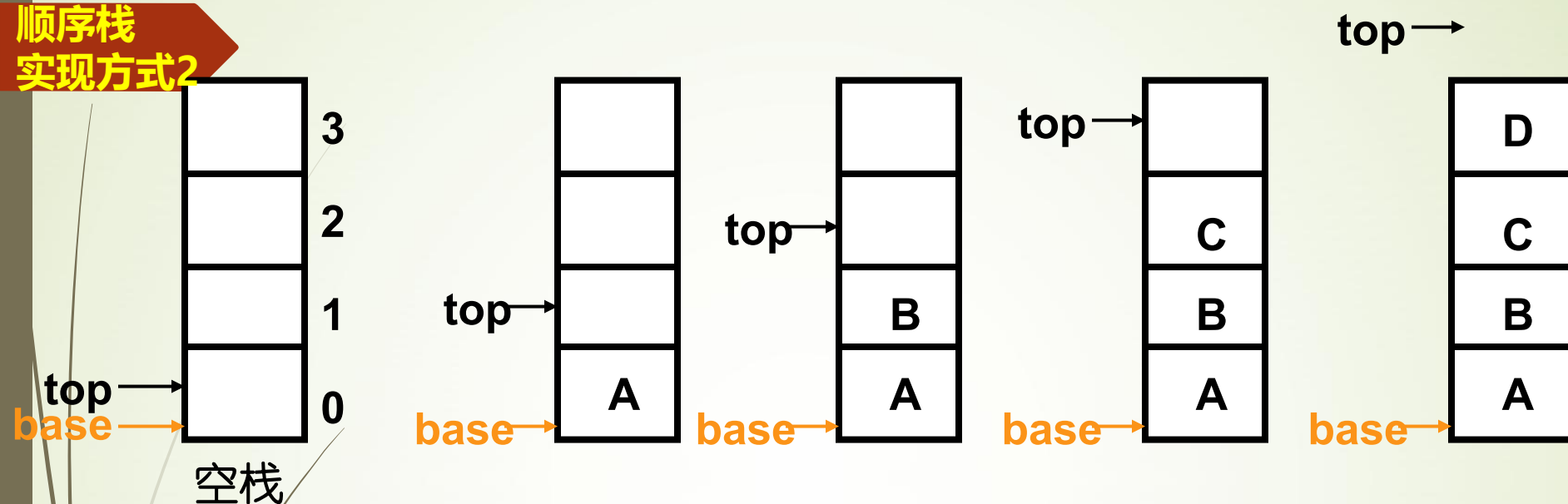
弹出: $x = \text{POP}()$

采用栈顶为top指针，top设在队尾+1

顺序栈的表示

top 指向栈顶元素之上的下标地址, $base$ 永远不动

顺序栈 实现方式2



$base == top$
 $(top - base) == 0$
是栈空标志

$top - base == MAXstacksize$
是栈满标志

栈满时的处理方法:

- 1、**报错**, 返回操作系统。
- 2、**分配更大的空间**作为栈的存储空间, 将原栈的内容移入新栈。

顺序栈的表示

顺序栈 实现方式2

```
#define MAXSIZE 100
typedef int SElemType;
typedef struct
{
    SElemType *base;
    SElemType *top;
    int stacksize;
}SqStack;
```

为什么不是这样的定义呢？

```
typedef struct
{
    unsigned int base;
    unsigned int top;
    SElemType *s;
    int stacksize;
}SqStack;
```

顺序栈的表示

顺序栈 实现方式2

```
#define MAXSIZE 100
typedef int SElemType;
typedef struct
{
    SElemType *base;
    SElemType *top;
    int stacksize;
} SqStack;
```

顺序栈初始化

```
Status InitStack( SqStack &S )
{
    S.base = new SElemType[MAXSIZE];
    if( !S.base ) return OVERFLOW;
    S.top = S.base;
    S.stackSize = MAXSIZE;
    return OK;
}
```

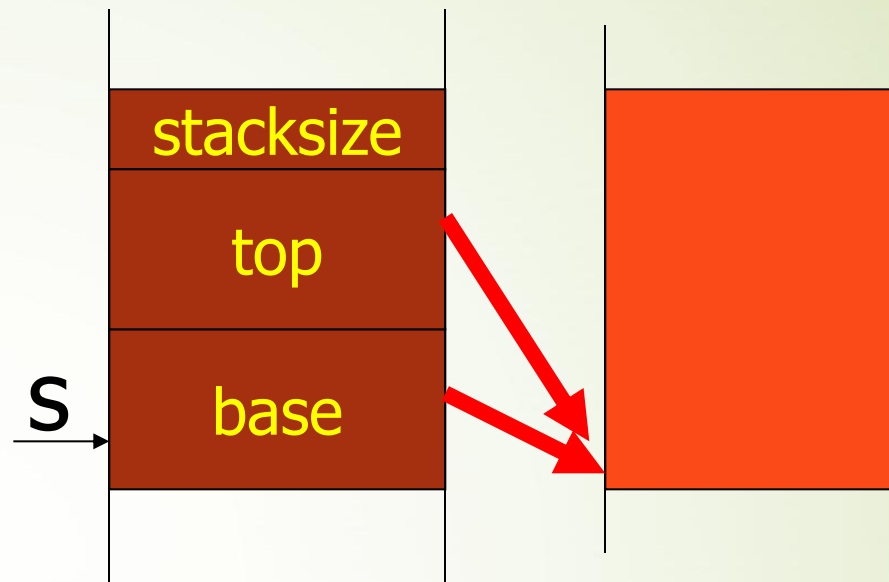
顺序表去哪儿了？
先看一下初始化程序

base指向了一个顺序表，
base永远不变
base就是这个顺序表
base[s.top-s.base]？
top指向的栈顶上面的那个位置

顺序栈初始化

顺序栈 实现方式2

- 初始化栈，即构造一个空栈的步骤：
 - 分配空间并检查空间是否分配失败，若失败则返回错误
 - 设置栈底和栈顶指针
 $S.top = S.base$;
 - 设置栈的大小



顺序栈初始化

顺序栈 实现方式2

```
//采用固定长度栈的数据存储结构
Status InitStack( SqStack &S )
{
    S.base = new SElemType[MAXSIZE];
    if( !S.base ) return OVERFLOW;
    S.top = S.base;
    S.stackSize = MAXSIZE;
    return OK;
}
```

判断顺序栈是否为空

顺序栈 实现方式2

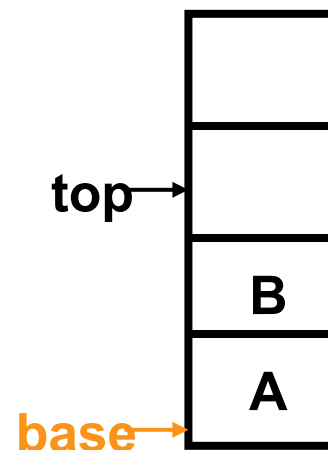
```
//采用固定长度栈的数据存储结构
bool StackEmpty( SqStack S )
{
    if(!S.base)
        return false;
    else
        return S.top == S.base;
}
```



求顺序栈的长度

顺序栈 实现方式2

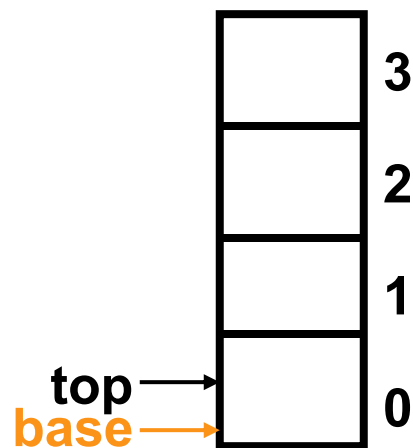
```
//采用固定长度栈的数据存储结构
int StackLength( SqStack S )
{
    if(!S.base)
        return 0;
    else
        return S.top - S.base;
}
```



清空顺序栈

顺序栈 实现方式2

```
//采用固定长度栈的数据存储结构
Status ClearStack( SqStack S )
{
    if( S.base )
    {
        S.top = S.base;
        return OK;
    }
    else
        return ERROR;
}
```



销毁顺序栈

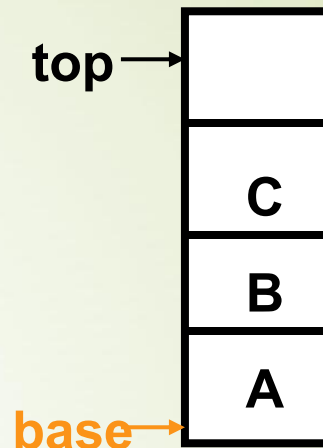
顺序栈 实现方式2

```
// 采用固定长度栈的数据存储结构
Status DestroyStack( SqStack &S )
{
    if( S.base )
    {
        delete S.base ;
        S.stacksize = 0;
        S.base = S.top = NULL;
    }
    return OK;
}
```

顺序栈进栈

顺序栈 实现方式2

- (1) 判断是否栈满，若满则出错
- (2) 元素e压入栈顶
- (3) 栈顶指针加1



```
Status Push( SqStack &S, SElemType e)
{
    if((!S.base)    //栈未初始化
        || (S.top-S.base==S.stacksize )) //栈满
        return ERROR;

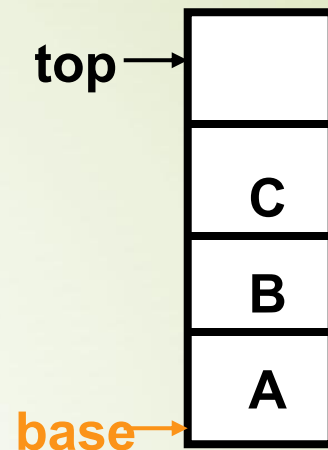
    *S.top++=e;
    return OK;
}
```

***S.top=e;**
S.top++;

顺序栈出栈

顺序栈 实现方式2

- (1) 判断是否栈空，若空则出错
- (2) 获取栈顶元素e
- (3) 栈顶指针减1



```
Status Pop( SqStack &S, SElemType &e)
{
    if( (!S.base)
        || (S.top == S.base )) // 栈空
        return ERROR;

    e = *--S.top;
    return OK;
}
```

--S.top;
e=*S.top;

取顺序栈栈顶元素

顺序栈 实现方式2

- (1) 判断是否空栈，若空则返回错误
- (2) 否则通过栈顶指针获取栈顶元素



```
Status GetTop( SqStack S, SElemType &e)
{
    if((!S.base)
        || (S.top == S.base) )    // 栈空
        return ERROR;

    e = *( S.top - 1 );
    return OK;
}
```

$e = *(S.top --);$
行不行?
取==读取,
不是弹出, pop

练习

顺序栈 实现方式2

在一个具有 n 个单元的顺序栈中，假设以地址高端作为栈底，以 top 作为栈顶指针，则当作进栈处理时， top 的变化为

D

- A. top 不变
- B. $top=0$
- C. $top++$
- D. $top--$

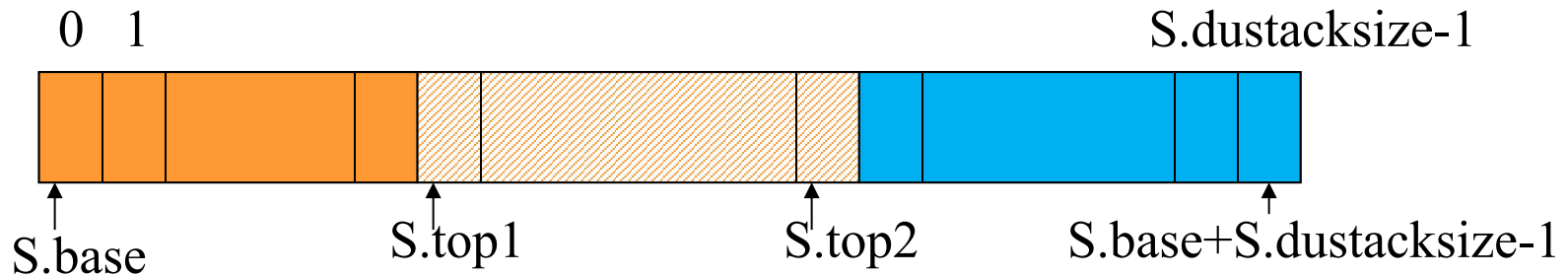
两个栈共享一组地址连续的存储单元

顺序栈
实现方式

[类型定义] 数组(栈空间) + 两个栈顶指示

```
#define DUSTACK_SIZE 100 //两栈的总允许容量
typedef struct {
    SElemType *base;
    SElemType * top1, * top2;
    int dustacksize ;
}DuStack
```

DuStack S;



链栈的表示

栈的链式实现

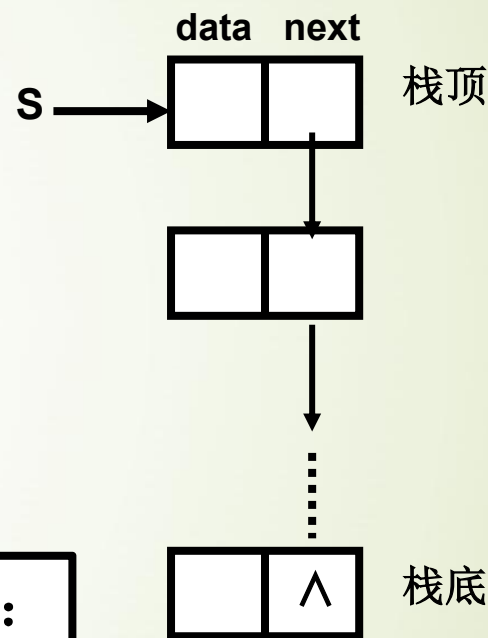
- ✓ 运算是受限的单链表，只能在链表头部进行操作，故没有必要附加头结点。栈顶指针就是链表的头指针

```
typedef struct StackNode
{
    SElemType data;
    struct StackNode *next;
}StackNode, *LinkStack;
LinkStack S;
```

链表的结点定义如下（和stack是一样的）：

```
typedef struct LNode
{
    ElemType data;
    struct LNode *next;
} LNode, *LinkList;
```

定义链表头的指针变量： LinkList H;



链栈的初始化

栈的 链式实现

$S \longrightarrow \wedge$

```
void InitStack(LinkStack &S )  
{  
    S = NULL;  
}
```

就是一个没有头结点的单链表！

判断链栈是否为空

栈的 链式实现

```
Status StackEmpty(LinkStack S)
{
    if (S==NULL) // 常写成 if(!S)
        return OK;
    else
        return ERROR;
}
```

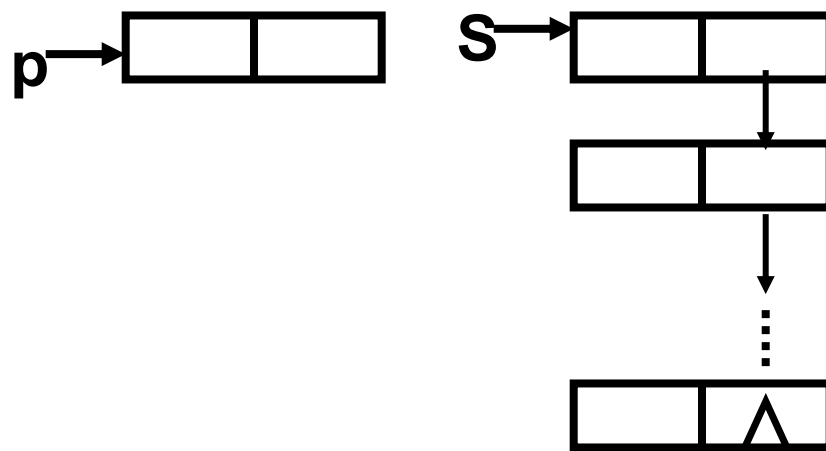
```
bool StackEmpty(LinkStack S)
{
    return !S
}
```

链栈进栈

栈的 链式实现

```
Status Push(LinkStack &S, SElemType e)
{
    p=new StackNode;           //生成新结点p
    if (!p)
        return OVERFLOW;

    p->data=e;
    p->next=S;
    S=p;
    return OK;
}
```

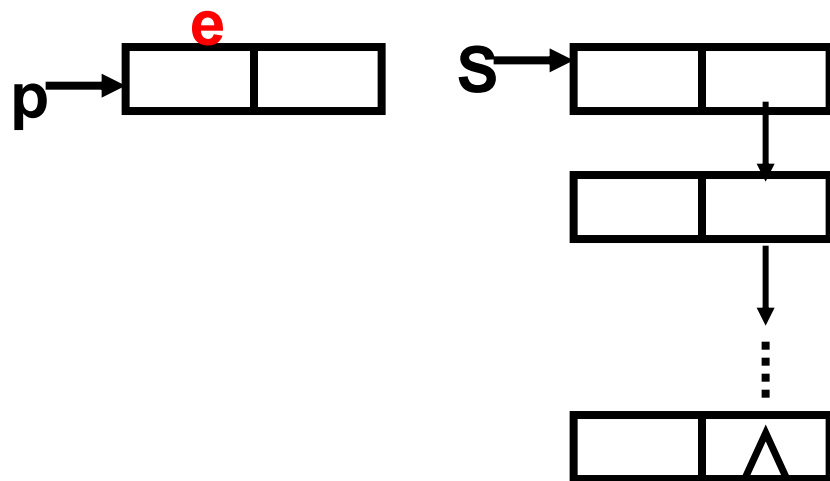


链栈进栈

栈的 链式实现

```
Status Push(LinkStack &S , SElemType e)
{
    p=new StackNode;           //生成新结点p
    if (!p)
        return OVERFLOW;

    p->data=e;
    p->next=S;
    S=p;
    return OK;
}
```

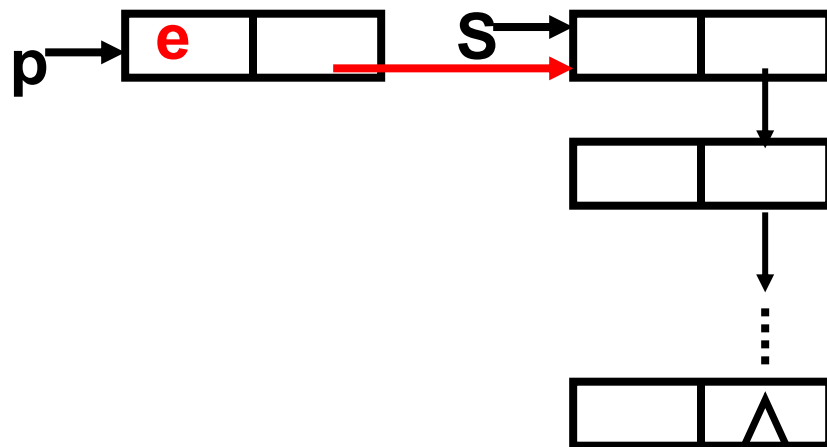


链栈进栈

栈的 链式实现

```
Status Push(LinkStack &S , SElemType e)
{
    p=new StackNode;           //生成新结点p
    if (!p)
        return OVERFLOW;

    p->data=e;
    p->next=S;
    S=p;
    return OK;
}
```



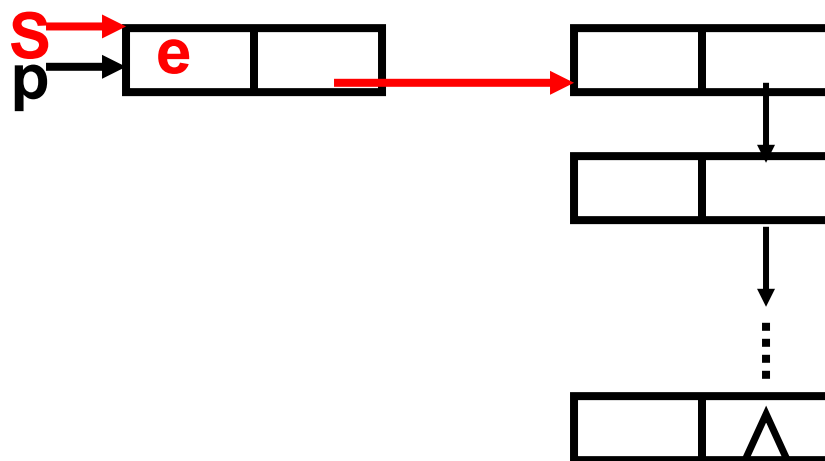
链栈进栈

栈的 链式实现

就是一个没有头结点的单链表的头插入算法！

```
Status Push(LinkStack &S , SElemType e)
{
    p=new StackNode;           //生成新结点p
    if (!p)
        return OVERFLOW;

    p->data=e;
    p->next=S;
    S=p;
    return OK;
}
```



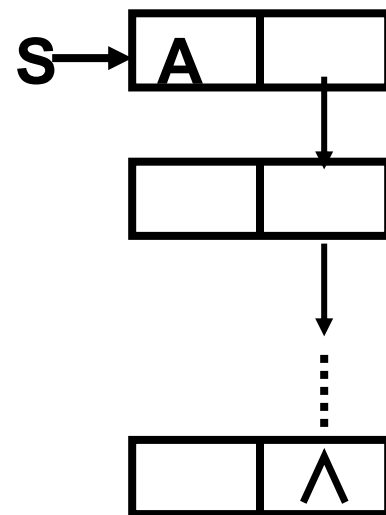
链栈出栈

栈的 链式实现

```
Status Pop(LinkStack &S, SElemType &e)
{
    if (S==NULL)
        return ERROR;

    e = S->data;
    p = S;
    S = S->next;
    delete p;
    return OK;
}
```

e = 'A'

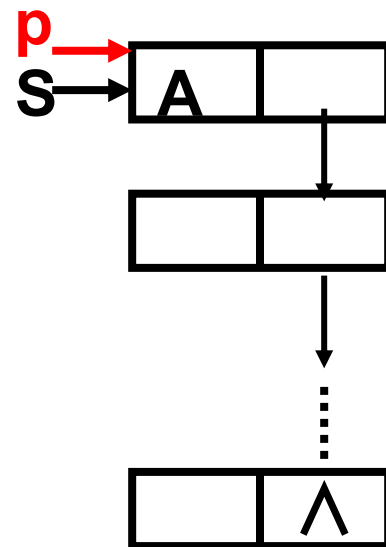


链栈出栈

栈的 链式实现

```
Status Pop(LinkStack &S, SElemType &e)
{
    if (S==NULL)
        return ERROR;
    e = S->data;
    p = S;
    S = S->next;
    delete p;
    return OK;
}
```

$e = 'A'$

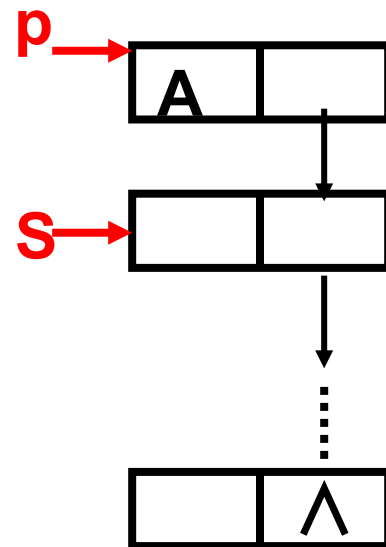


链栈出栈

栈的链式实现

```
Status Pop(LinkStack &S, SElemType &e)
{
    if (S==NULL)
        return ERROR;
    e = S-> data;
    p = S;
    S = S-> next;
    delete p;
    return OK;
}
```

e = 'A'



链栈出栈

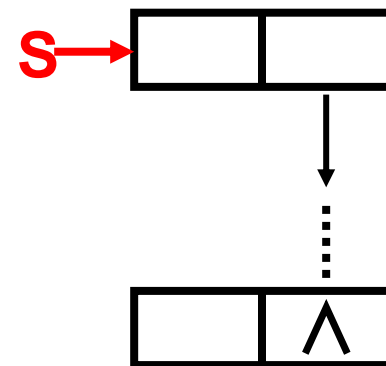
栈的 链式实现

就是一个没有头结点的单链表的头删除算法！

```
Status Pop(LinkStack &S, SElemType &e)
{
    if (S==NULL)
        return ERROR;

    e = S->data;
    p = S;
    S = S->next;
    delete p;
    return OK;
}
```

e = 'A'



取链栈栈顶元素

栈的 链式实现

```
SElemType GetTop(LinkStack S)
{
    if (S==NULL)
        return 1;
    else
        return S->data;
}
```

3.5 队列的表示和操作的实现

练习

设栈S和队列Q的初始状态为空，元素e1、e2、e3、e4、e5和e6依次通过S，一个元素出栈后即进入Q，若6个元素出队的序列是e2、e4、e3、e6、e5和e1，则栈S的容量至少应该是（ ） **B**

(A) 2

(B) 3

(C) 4

(D) 6

队列的抽象数据类型

队列的
实现

ADT Queue {

数据对象: $D = \{a_i \mid a_i \in ElemSet, i = 1, 2, \dots, n, n \geq 0\}$

数据关系: $R_1 = \{ \langle a_{i-1}, a_i \rangle \mid a_{i-1}, a_i \in D, i = 1, 2, \dots, n \}$

基本操作: 约定 a_1 端为队列头, a_n 端为队列尾

- (1) InitQueue (&Q) //构造空队列
- (2) DestroyQueue (&Q) //销毁队列
- (3) ClearQueue (&S) //清空队列
- (4) QueueEmpty(S) //判空. 空--TRUE,

队列的抽象数据类型

队列的实现

(5) QueueLength(Q) //取队列长度

(6) GetHead (Q,&e) //取队头元素,

(7) EnQueue (&Q,e) //入队列

(8) DeQueue (&Q,&e) //出队列

(9) QueueTraverse(Q,visit()) //遍历

}ADT Queue

队列的顺序表示——用顺序存储的线性表

顺序存储的 队列

```
#define MAXQSIZE 100    //最大队列长度  
typedef struct {  
    QElemType *base;    //初始化的动态分配存储空间  
    int front;           //头指针  
    int rear;            //尾指针  
} SqQueue;
```

//头指针

这里的两个指针
是用顺序表的下
标，即位置号来
做的。why?

顺序存储的队列和顺序存储的栈的对比:

顺序存储的队列

```
#define MAXQSIZE 100 //最大队列长度
typedef struct {
    QElemType *base; //初始化的动态分配存储空间
    int front;        //头指针
    int rear;         //尾指针
}SqQueue;
```

为什么顺序存储的队列用整数做头尾指针?

为什么顺序存储的栈用指针做栈顶和栈底的指针?

在循环队列时用取模运算, 计算绕回来的队头和队尾的位置方便。

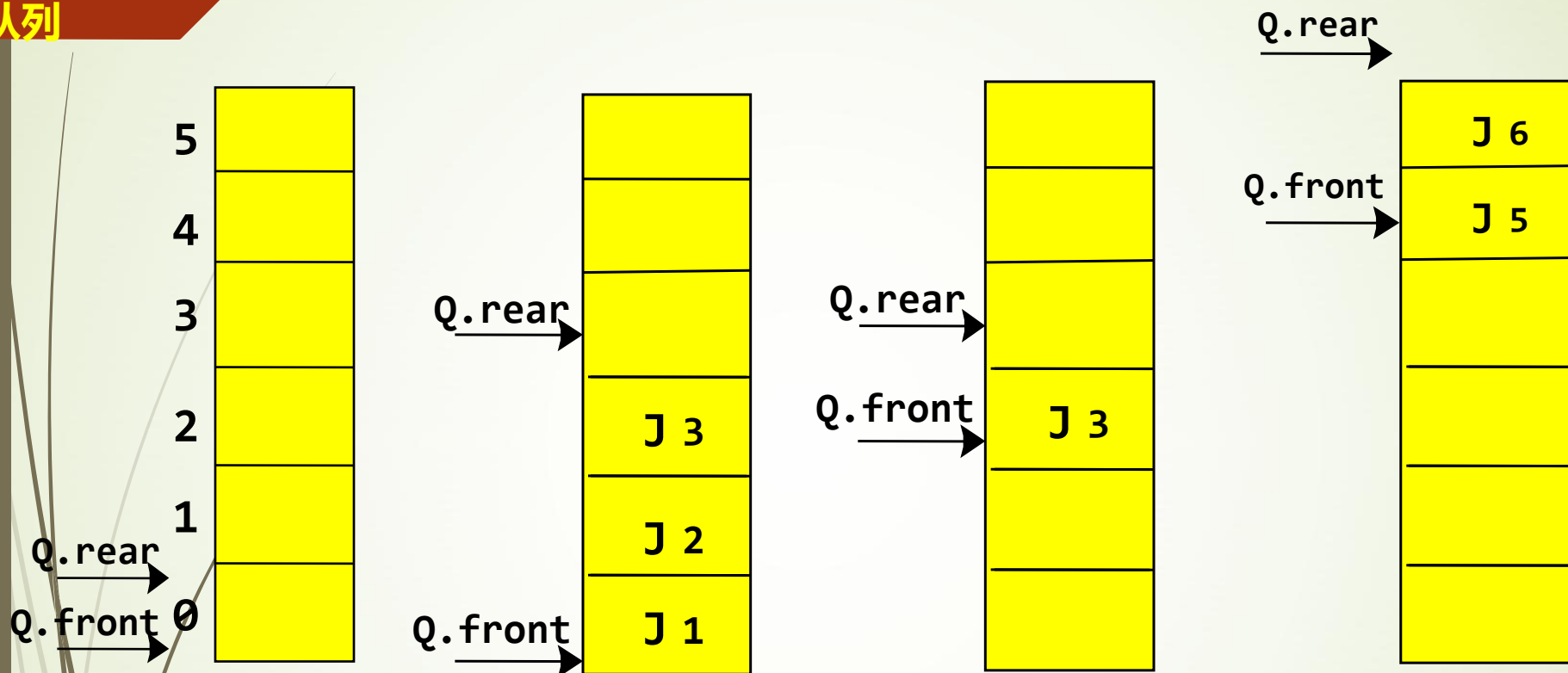
在顺序栈时对地址值 (指针值) 用++、--计算快捷

顺序栈的表示

```
#define MAXSIZE 100
typedef int SElemType;
typedef struct
{
    SElemType *base;
    SElemType *top;
    int stacksize;
}SqStack;
```

队列的顺序表示——用线性表

顺序存储的队列



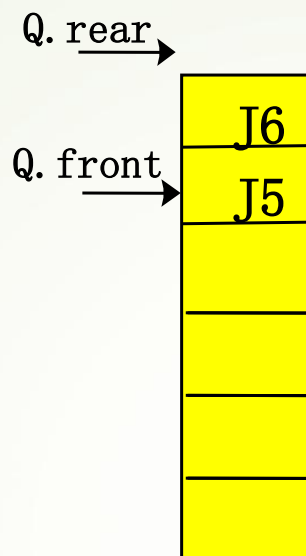
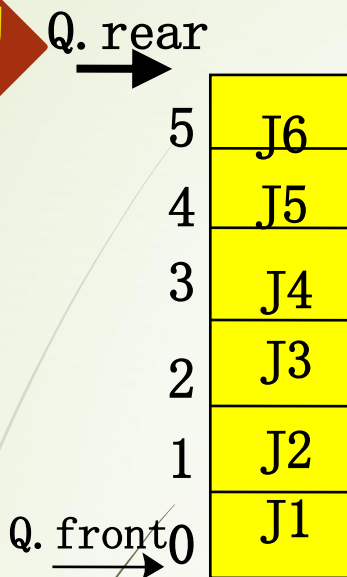
初始:
 $front=rear=0$

空队标志: $front==rear$
入队: $base[rear++]=x;$
出队: $x=base[front++];$

存在的问题

设队列大小为M

顺序存储的
队列



front=0
rear=M时
再入队——真溢出

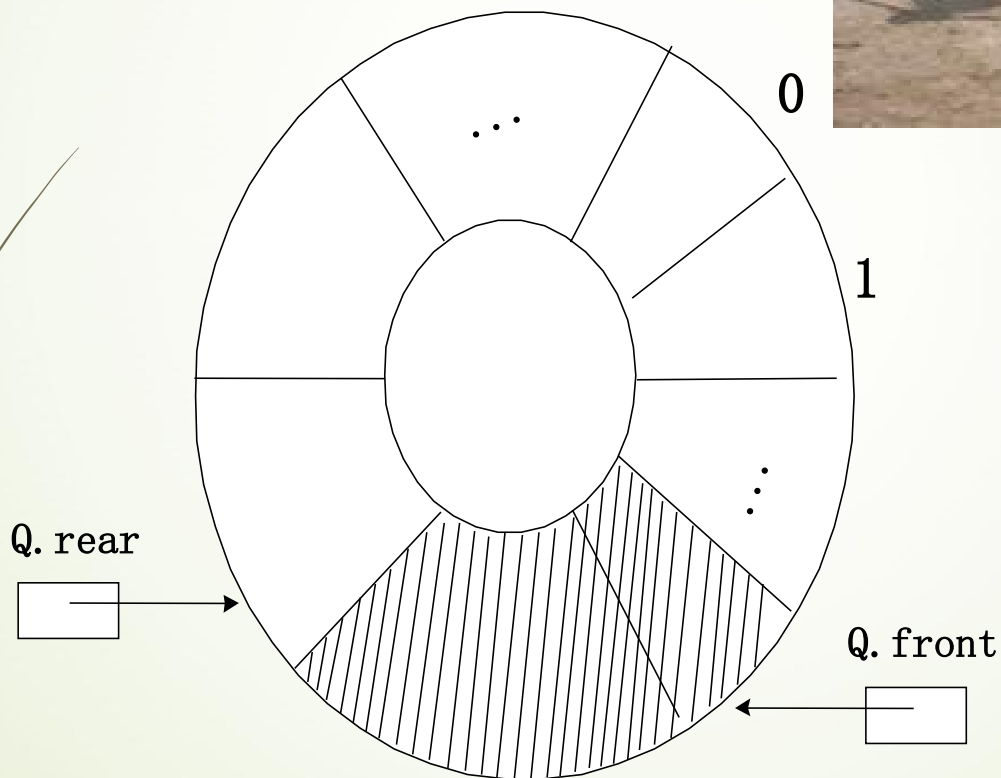
front≠0
rear=M时
再入队——假溢出



解决的方法——循环队列

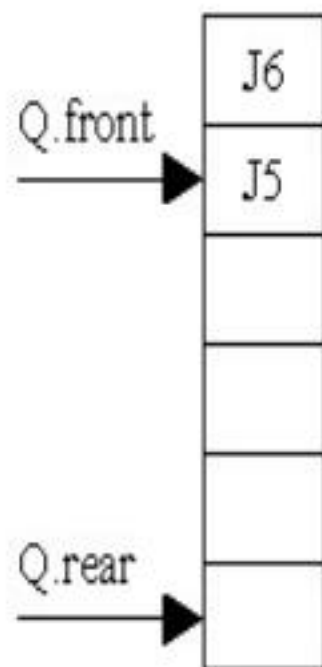
解决的方法——循环队列

顺序存储的
队列

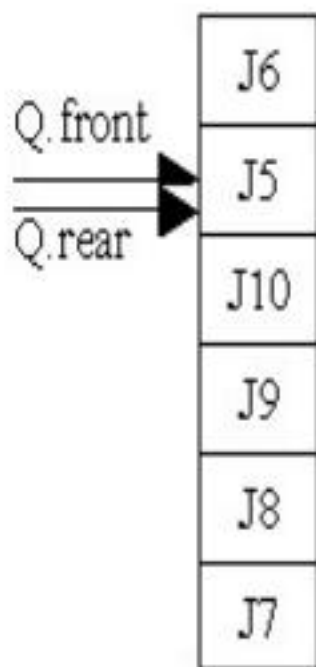


解决的方法——循环队列

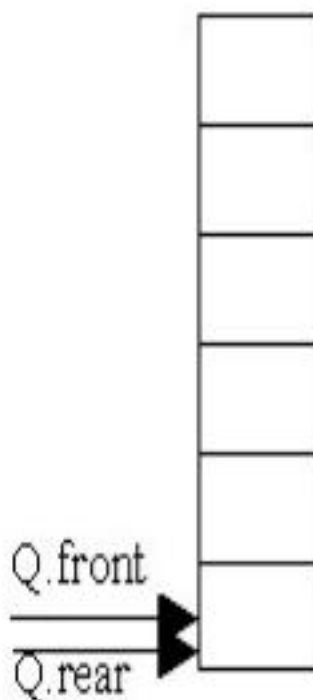
顺序存储的 队列



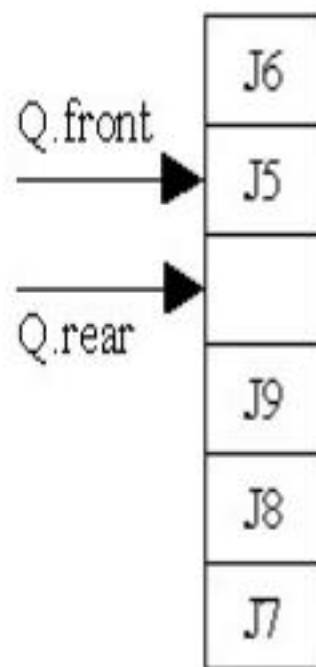
(a) 一般情况



(b) 队列空间被占满



(c) 空队



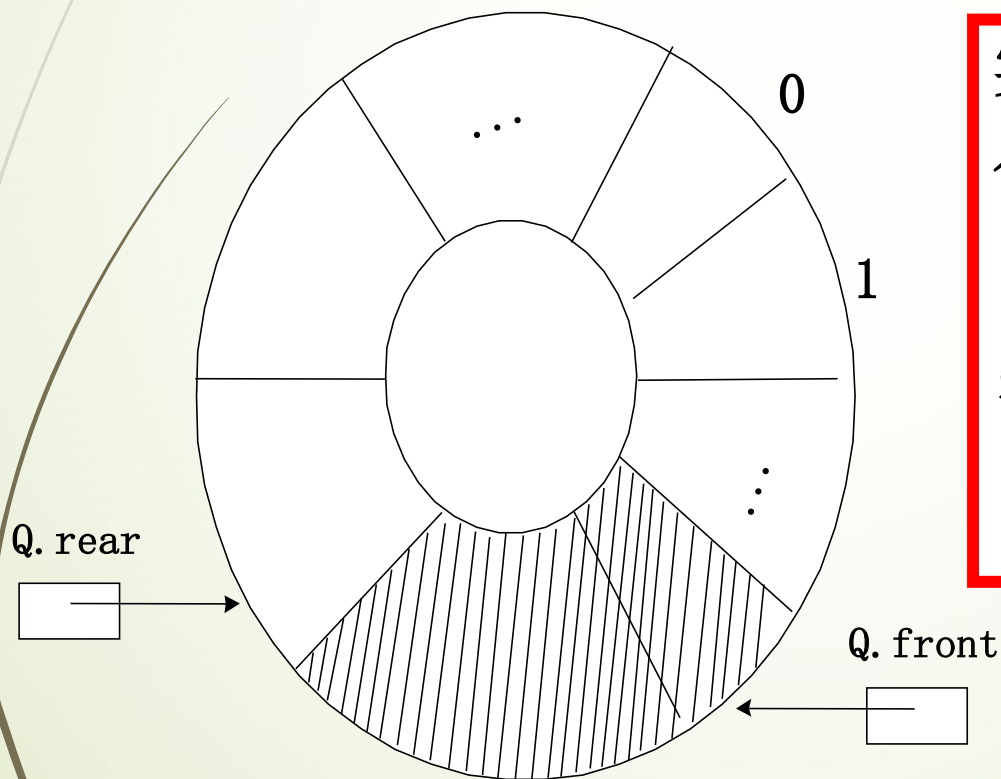
(d) 呈“满”状态的循环队列

图 3.12 循环队列中头、尾指针和元素之间的关系

解决的方法——循环队列

顺序存储的
队列

**base[0]接在base[M-1]之后
若 $\text{rear}+1==M$
则令 $\text{rear}=0$;**



实现：利用“模”运算
入队：

**$\text{base}[\text{rear}]=x;$
 $\text{rear}=(\text{rear}+1)\%M;$**

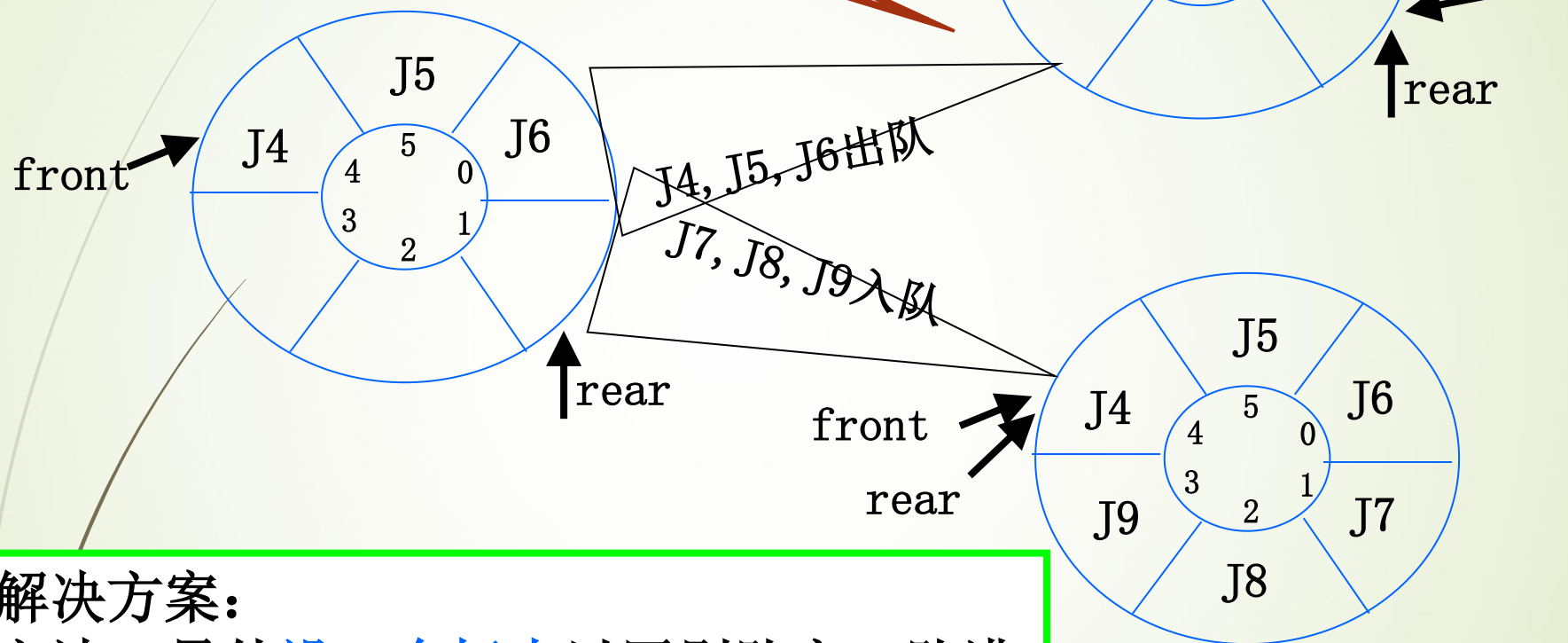
出队：

**$x=\text{base}[\text{front}];$
 $\text{front}=(\text{front}+1)\%M;$**



顺序存储的
队列

队空: $front == rear$
队满: $front == rear$



解决方案:

方法1. 另外设一个标志以区别队空、队满

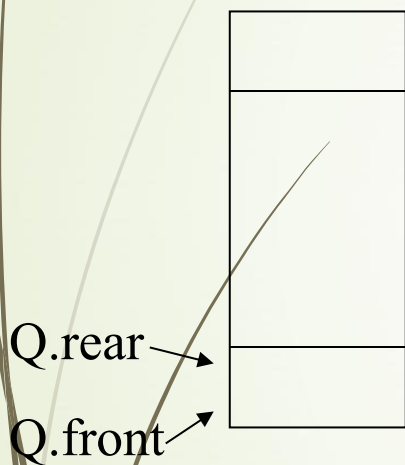
方法2. 少用一个元素空间 (教材采用):

队空: $front == rear$

队满: $(rear + 1) \% M == front$

循环队列的两种控制方式

顺序存储的 队列

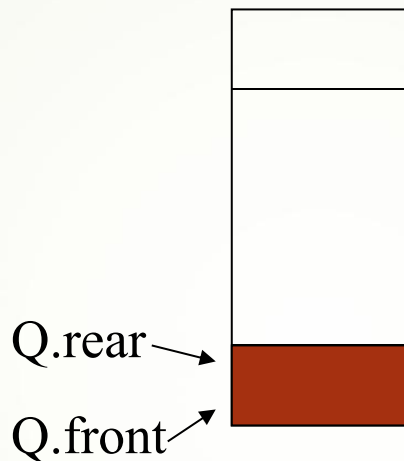


队空：
 $Q.front = Q.rear$

方式1:

需要另设一个标志位在
区别队空和队中存在元
素的状态。

```
typedef struct {  
    QElemType *base;  
    int front; //头指针  
    int rear; //尾指针  
    bool isQueueFull; // 记录队列是否为空  
                        //或定义一个记录队列长度变量  
} SqQueue;
```

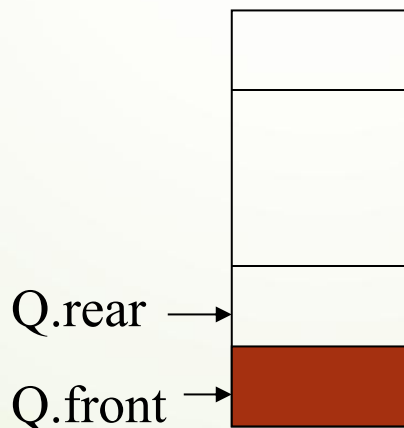


方式2:

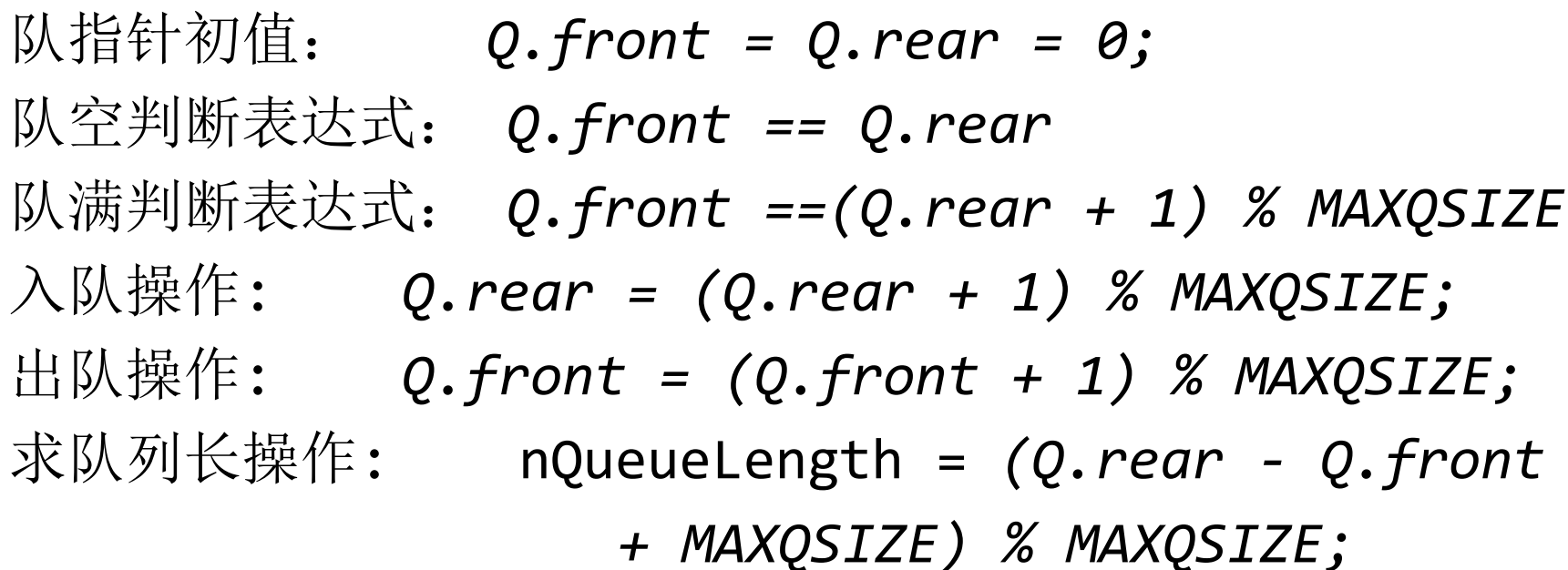
(本课程采用方式)

少用一个元素空间。

空间不能用满，否则无
法区别队列是空还是满。



顺序存储的队列



循环队列

顺序存储的
队列



```
#define MAXQSIZE 100 //最大长度
typedef struct {
    QElemType *base; //初始化的动态分配存储空间
    int front;        //头指针
    int rear;         //尾指针
} SqQueue;
```

循环队列初始化

顺序存储的 队列

```
Status InitQueue (SqQueue &Q)
{
    Q.base = new QElemType[MAXQSIZE]
    if(!Q.base)
        return OVERFLOW;
    Q.front=Q.rear=0;
    return OK;
}
```

求循环队列的长度

顺序存储的
队列

```
Int QueueLength(SqQueue Q)
{
    return (Q.rear - Q.front + MAXQSIZE)
        % MAXQSIZE;
}
```

循环队列入队

顺序存储的 队列

```
Status EnQueue(SqQueue &Q, QElemType e)
{
    if( (Q.rear+1) % MAXQSIZE == Q.front )
        return ERROR;

    Q.base[Q.rear] = e;
    Q.rear = (Q.rear + 1) % MAXQSIZE;
    return OK;
}
```

循环队列出队

顺序存储的 队列

```
Status DeQueue (SqQueue &Q, QElemType &e)
{
    if(Q.front == Q.rear)
        return ERROR;
    e=Q.base[Q.front];
    Q.front=(Q.front+1)%MAXQSIZE;
    return OK;
}
```

循环队列出队

顺序存储的 队列

```
Status DeQueue (SqQueue &Q, QElemType &e)
{
    if(Q.front != Q.rear)
    {
        e = Q.base[Q.front];
        Q.front = (Q.front+1) % MAXQSIZE;
        return OK;
    }
    return ERROR;
}
```

顺序存储队列的几种实现方案对比：

顺序存储的 队列

1、队尾固定为0下标的元素

缺点：入队列时需向后移动队列中的剩余元素

2、队头固定为0下标的元素

缺点：出队列时需向前移动队列中的剩余元素

3、队头队尾均不固定

优点：入出队列均无需移动元素

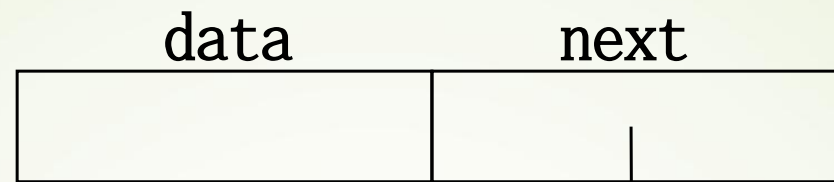
缺点：假溢出

4、循环队列

链队列

链式存储的
队列

Q.front



队头



⋮

Q.rear



队尾

链队列

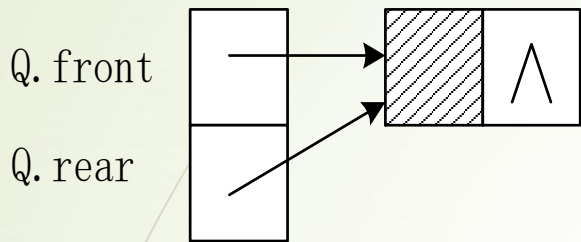
链式存储的 队列

```
typedef struct QNode{
    QElemType      data;
    struct Qnode   * next;
}Qnode, *QueuePtr;

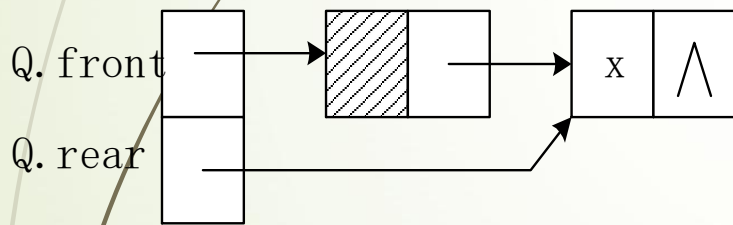
typedef struct {
    QueuePtr  front;           //队头指针
    QueuePtr  rear;           //队尾指针
    int  nQueueLength;        //队列当前长度
}LinkQueue;
LinkQueue  Q; //有头尾指针的单链表
```

链队列

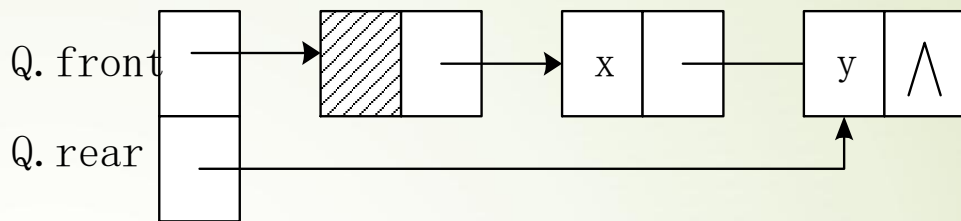
链式存储的队列



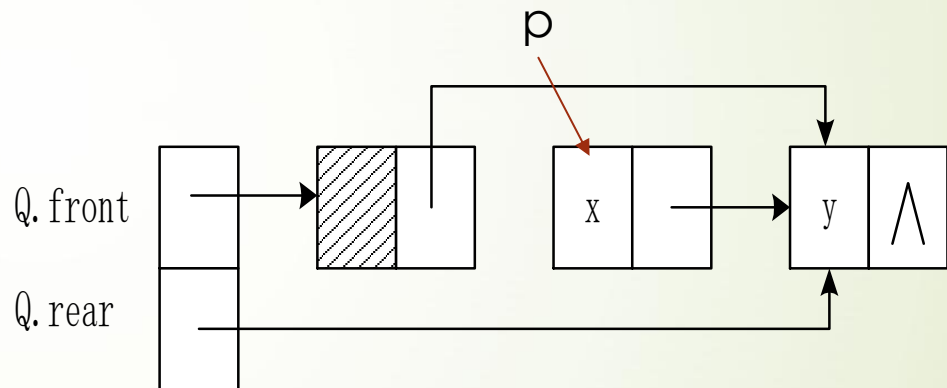
(a) 空队列



(b) 元素x入队列



(c) 元素y入队列



(d) 元素x出队列

就是一个有头结点的单链表,用front和rear分别指向头和尾

链队列初始化

链式存储的 队列

```
Status InitQueue (LinkQueue &Q)
{
    //申请头结点空间
    Q.front=Q.rear=(QueuePtr)m1loc(sizeof(QNode));
    if(!Q.front)
        return OVERFLOW;
    //队列为空
    Q.front->next=NULL;
    Q.nQueueLength = 0;
    return OK;
}
```

销毁链队列

链式存储的 队列

```
Status DestroyQueue (LinkQueue &Q)
{
    QueuePtr p=Q.front;
    while(p)
    {
        Q.rear = p->next; //把Q.rear当做临时变量q了
        free(p);
        p=Q.rear;
    }
    Q.front=Q.rear=NULL;
    Q.nQueueLength = 0;
    return OK;
}
```

判断链队列是否为空

链式存储的 队列

```
Status QueueEmpty (LinkQueue Q)
{
    if(Q.front && Q.rear)
        return (Q.front==Q.rear);
    return ERROR;
}
```

//另外一个方式

```
Status QueueEmpty (LinkQueue Q)
{
    return (!Q.nQueueLength);
}
```

求链队列的队头元素

链式存储的 队列

```
Status GetHead(LinkQueue Q, QElemType &e)
{
    if(!Q.front || !Q.rear)
        return ERROR;
    if(Q.front==Q.rear)
        return ERROR;
    e=Q.front->next->data;
    return OK;
}
```

链队列入队

链式存储的 队列

```
Status EnQueue(LinkQueue &Q, QElemType e)
```

```
{
```

```
    if(!Q.front || !Q.rear)
```

```
        return ERROR;
```

```
    p=(QueuePtr)malloc(sizeof(QNode));
```

```
    if(!p)
```

```
        return OVERFLOW;
```

```
    p->data=e; p->next=NULL;
```

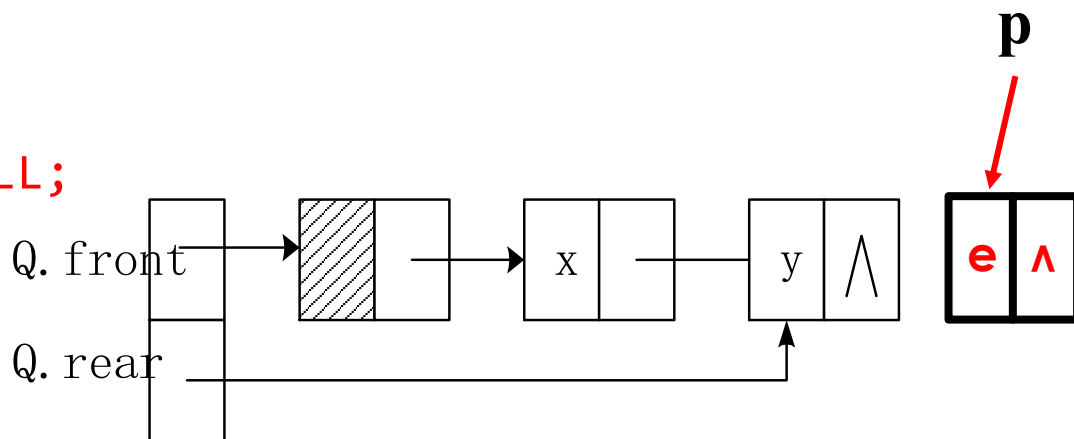
```
    Q.rear->next=p;
```

```
    Q.rear=p;
```

```
    Q.nQueueLength++;
```

```
    return OK;
```

```
}
```



就是一个有头结点的单链表的在
Q.rear之后插入的算法。

链队列出队

链式存储的 队列

```
Status DeQueue (LinkQueue &Q, QElemType &e)
```

```
{  
    if(!Q.front || !Q.rear)  
        return ERROR;  
    if(Q.front==Q.rear) //队列空  
        return ERROR;
```

```
    p=Q.front->next;
```

```
    e=p->data;
```

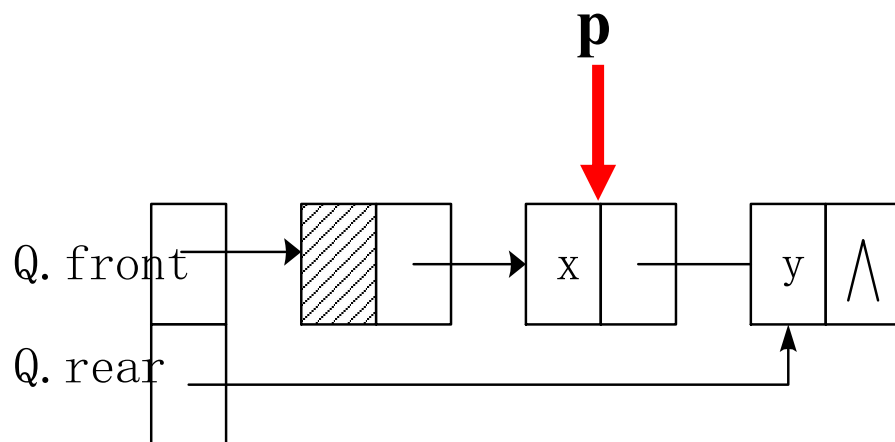
```
    Q.front->next=p->next;
```

```
    if(Q.rear==p) Q.rear=Q.front; //删除p后, Q.rear也失效了; 此时队列空  
    delete p;
```

```
    Q.nQueueLength--;
```

```
    return OK;
```

```
}
```



就是一个有头结点的单链表的头删除的算法。

思考题：

- 1、为什么顺序存储的队列要用循环方式来合理利用存储空间，而栈则无需考虑用循环方式合理利用存储空间这个问题？
- 2、能否用2个栈来实现1个队列？

利用线性表的基本方法（函数）实现后的讨论

	栈		队列	
	栈顶为表头	栈顶为表尾	队头为表尾	队头为表头
顺序存储	PUSH, 在下标为1的位置插入, 顺序后移其他元素 时间复杂度$O(n)$ POP, 在下标为1的位置删除, 顺序前移其他元素 时间复杂度$O(n)$	PUSH, 用下标直接访问线性表的表尾 时间复杂度$O(1)$ POP, 用下标直接访问线性表的表尾 时间复杂度$O(1)$	EnQueue, 在下标为1的位置插入, 顺序后移其他元素 时间复杂度$O(n)$ DeQueue 用下标直接访问访问表尾 时间复杂度$O(1)$	EnQueue, 用下标直接访问访问表尾, 时间复杂度$O(1)$ DeQueue 在下标为1的位置删除, 顺序前移其他元素 时间复杂度$O(n)$
链式存储	PUSH, 头插法 时间复杂度$O(1)$ POP, 头指针后面删除 时间复杂度$O(1)$	PUSH, 尾插法, 先移动到表尾 时间复杂度$O(n)$ POP, 尾删除, 先移动到表尾前, 时间复杂度$O(n)$	EnQueue, 头插法 时间复杂度$O(1)$ DeQueue 尾删除, 先移动到表尾前, 时间复杂度$O(n)$	EnQueue, 尾插法, 先移动到表尾, 时间复杂度$O(n)$ DeQueue 头指针后面删除 时间复杂度$O(1)$

利用线性表的基本方法（函数）实现后的讨论

栈

栈顶为表头

顺序
存储

不采用此存储
方式

因时间复杂度
为 $O(n)$

栈顶为表尾

已是优化
结构

时间复杂
度均为
 $O(1)$

队列

队头为表尾

优化为循环表

使用队尾指针 $rear$ 和队头指针 $front$

1) 实现按 $rear$ 下标 $++$ 取模的直接加入
 $EnQueue$

2) 实现按 $front$ 下标 $-$ 取模的直接删除
 $DeQueue$

2) 无需移动表中已有元素同时也维持原
有元素间的顺序

时间复杂度均为 $O(1)$

队头为表头

链式
存储

已是优化结
构

时间复杂
度均为 $O(1)$

不采用此存储
方式

因时间复杂度
为 $O(n)$

优化为头尾双指针链表

使用队尾指针 $rear$ 和队头指针 $front$

1) $EnQueue$ 插入，在 $rear$ 指针后的
插入结点操作

2) $DeQueue$ ，对 $front$ 指针后结点的
删除操作

时间复杂度均为 $O(1)$